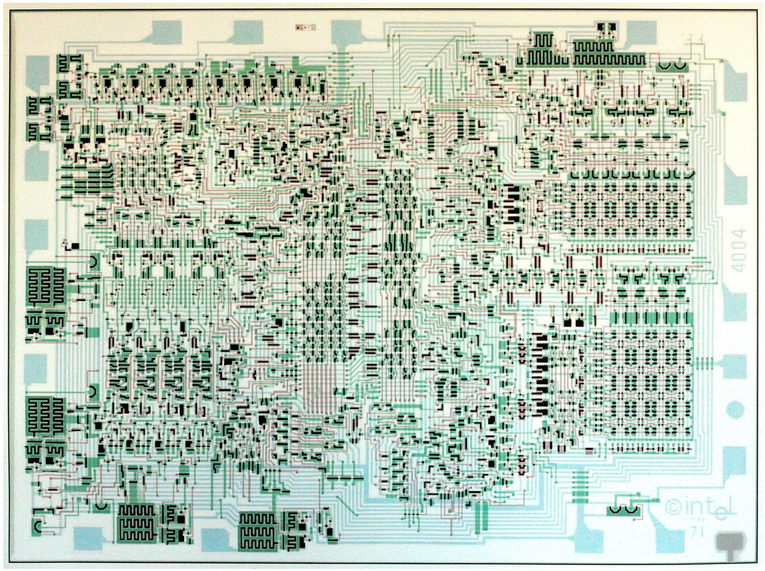
# AI for Chip Design: Unlocking New Frontiers in Automation and Scaling

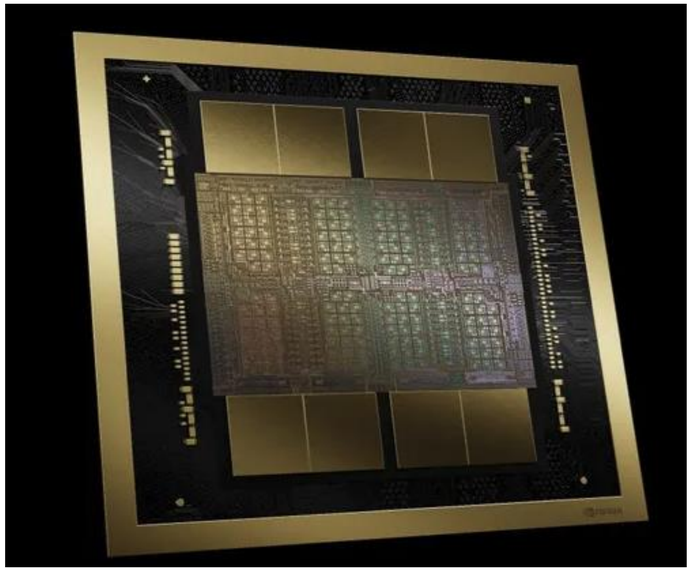
Haoxing (Mark) Ren

# Design Automation: Automate and Scale Human Efforts

100 Millions times scaling in transistor count and 200 Billion times scaling in compute




**Intel 4004 (1971)**
2300 transistors
100K 4B instruction/sec



**NVIDIA B200 (2024)**
208 Billion Transistors
20 Peta FP4/sec

# Examples of EDA Automation and Scaling

1973: SPICE, automate circuit simulation

1986: Synopsys Design Compiler, automate logic synthesis

1989: Cadence Verilog-XL, automate RTL simulation

1997: Cadence Silicon Ensemble , automate placement & route

1998: Synopsys PrimeTime , automate static timing analysis

2004: Apache RedHawk, automate power grid analysis, scaling with multi-threading

2008: Synopsys PrimeTime , scaling STA with multi-threading

2012: Cadence Innovus, scaling P&R with extensive multi-threading

2016: Ansys RedHawk-SC, scaling power grid analysis with distributed many cores

# What have been done in AI for Chip Design

Leverage AI to automate some difficult manual tasks

Various AI for Chip Design prediction work

- Congestion, IR drop, testability, etc
- Use supervised learning and CNN/GNN models

AI for PPA optimization

- Google's RL macro placement work, NVIDIA's RL for Prefix Adder optimization
- Find a better macro placement solution
- Using Reinforcement Learning and CNN models

Commercial EDA AI solutions (DSO.AI, Cerebrus, …)

- Search the flow parameter space to find a better configuration
- Using Bayesian Optimization and similar techniques

Scopes of automation are limited, methods are not scalable ( increasing the runtime)

Are there better opportunities?

# Unleash the potential of AI for automation and scaling

AI can help scale the existing EDA automation

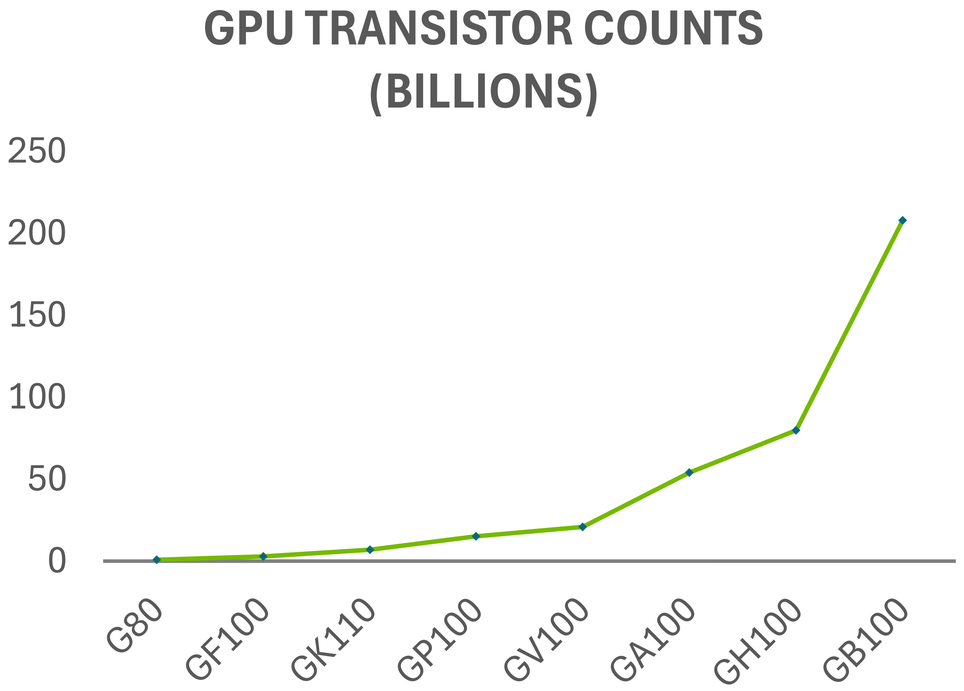
- Scaling physical design tools capacity
- DL framework, GenAI methodology, GPU accelerated differentiable optimization
- AI as a new way to leverage GPU power for EDA workloads

AI can automate more manual design tasks

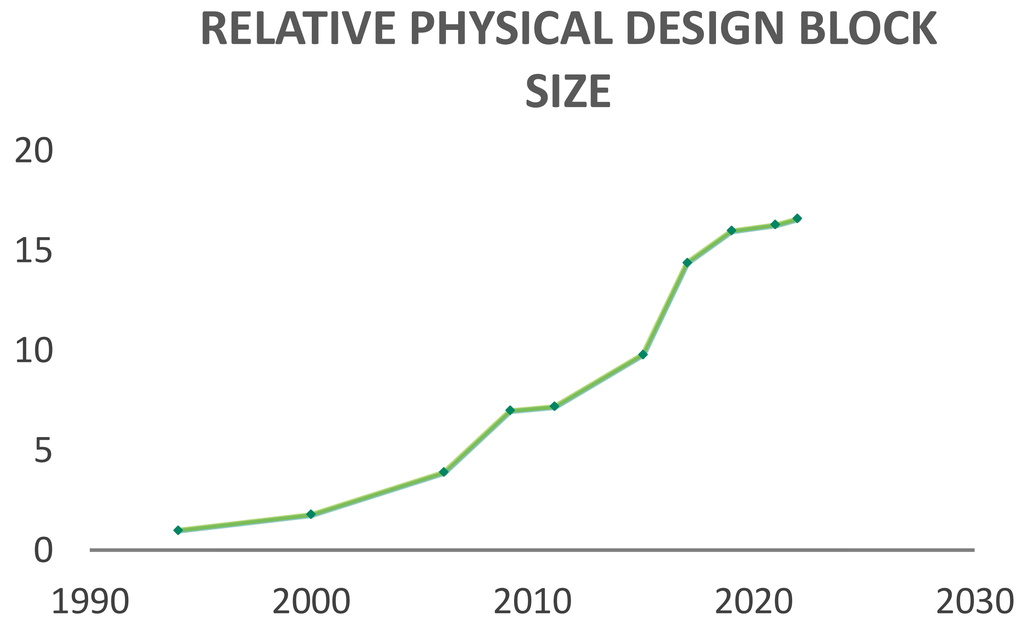
- Automate code writing and debug
- Automate QnA, design analysis, optimization and debug tasks
- Domain-adapted LLMs, Agentic systems
- AI as a new way to automate human effort

# Physical Design Scalability Challenge

## Mainly because of P&R tools scalability




Design size continue to scale




Average physical design block size levels off
>1 week turn around time for 1-2 Million cell block

*Sameer Halepete, Accelerating the Design and Performance of Next Generation Computing Systems with GPUs, ISPD'22*

# Accelerate Placement on GPU

## Nonlinear Placement Algorithm / Gradient-based Method

Minimize wirelength and density overflow

$$\min \quad f = WL(x, y) + \mu \sum_{bin} Overflow_{bin}(x, y)^2$$

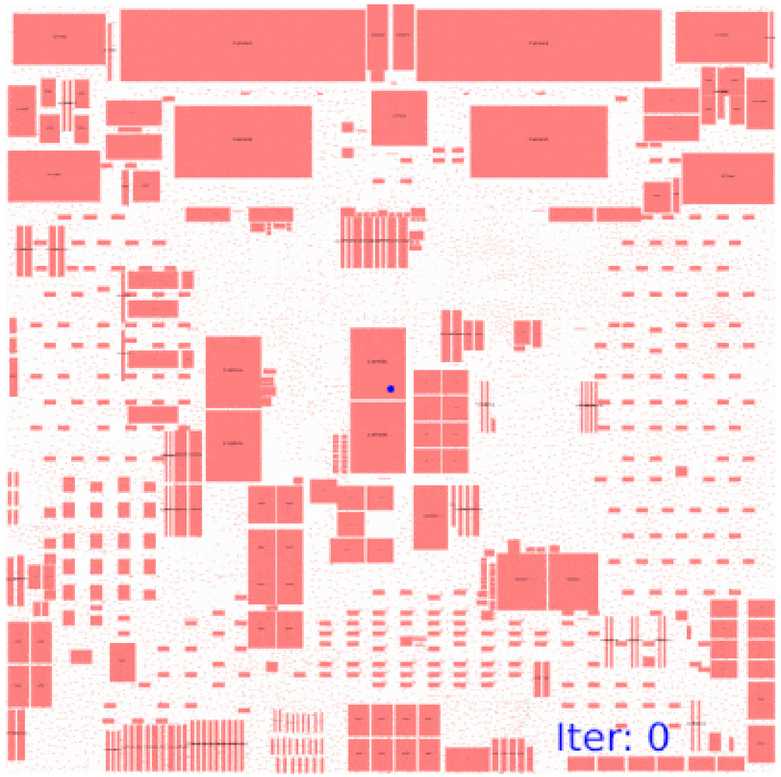
Gradient-based optimization :

$$x = x - \gamma \frac{\partial}{\partial x} f$$

$$y = y - \gamma \frac{\partial}{\partial y} f$$

Neural networks back propagation

$$\min \quad \sum_{i} loss(w, x_i)^2 + L_2(w)$$




Accelerate with DL Frameworks on GPU

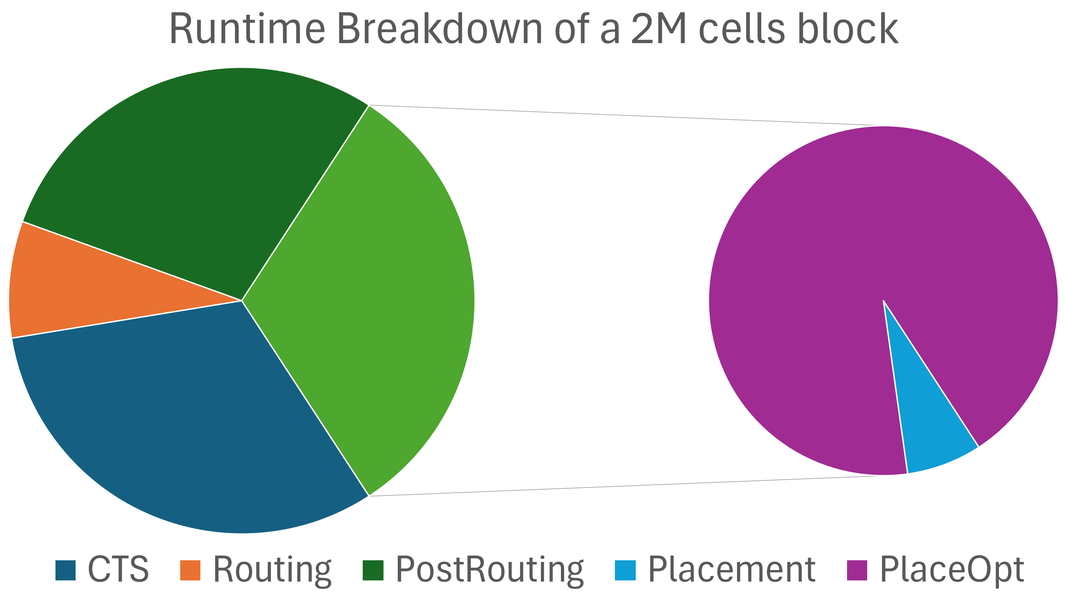
*H. Ren et al, Nonlinear Placement for VLSI using TensorFlow on GPU, GTC'18; DREAMPlace, DAC'19*

# Optimization is the Main Scalability Challenge

Optimization (gate sizing, buffering, etc. for better PPA) consumes the majority of the runtime in each major PD step

Why? Numerous heuristics to push for PPA limit, incremental timing, and routing updates

Is it optimal? Certainly not



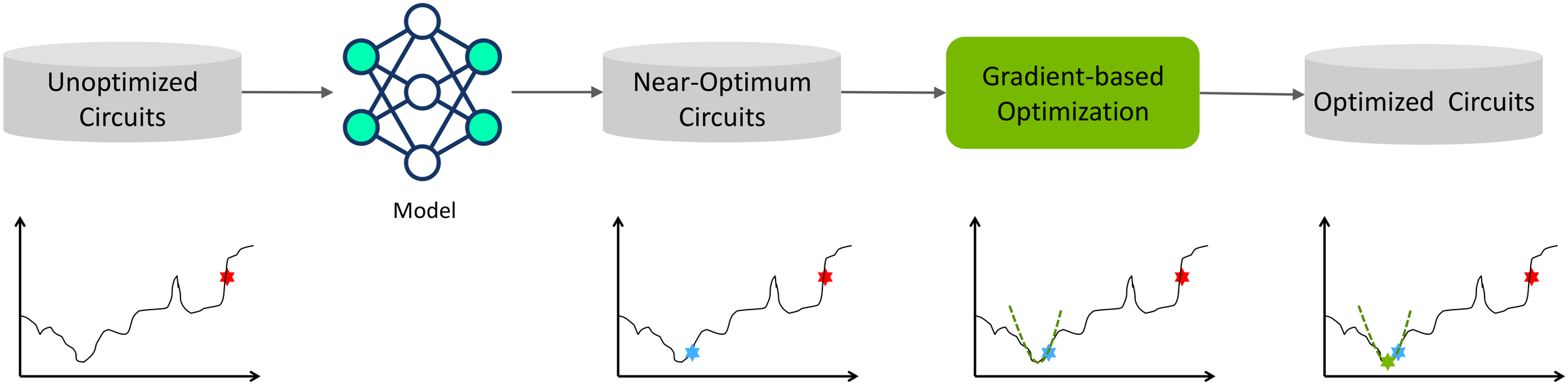
# AI Optimization Method Comparison

| | Advantage | Issues |
|---|---|---|
| Sequential Model-based Optimization (SMBO) | • Do not require gradients of the functions<br>• Easy to use, always work<br>• Commercial success | • Limited optimization variables (10s)<br>• Require repeated evaluations of functions (100s – 1000s) |
| RL | • Only needs reward evaluation, and requires no gradient from the reward<br>• Apply to dynamic/stochastic problems | • Limited optimization variables (100s)<br>• Require fast reward evaluation (secs)<br>• Long training time (hrs to days)<br>• Difficult to generalize |
| Generative AI | • No need to compute objective function and its gradient<br>• Scalable to millions of variables on GPUs | • Require a lot of optimized data<br>• Data collection challenge<br>• **How to deal with outliers** |
| Gradient-based Method | • Grounded optimization<br>• Scalable to millions of variables on GPUs | • Require differentiable objective functions<br>• Require relaxation and approximation<br>• **How to ensure global optimal** |

# Combine Gen AI and Gradient-Based Method

Leverage Gen AI model to generate a solution that is close to the global optimum

Build gradient-based optimization in the global optimum neighborhood to search for the final solution

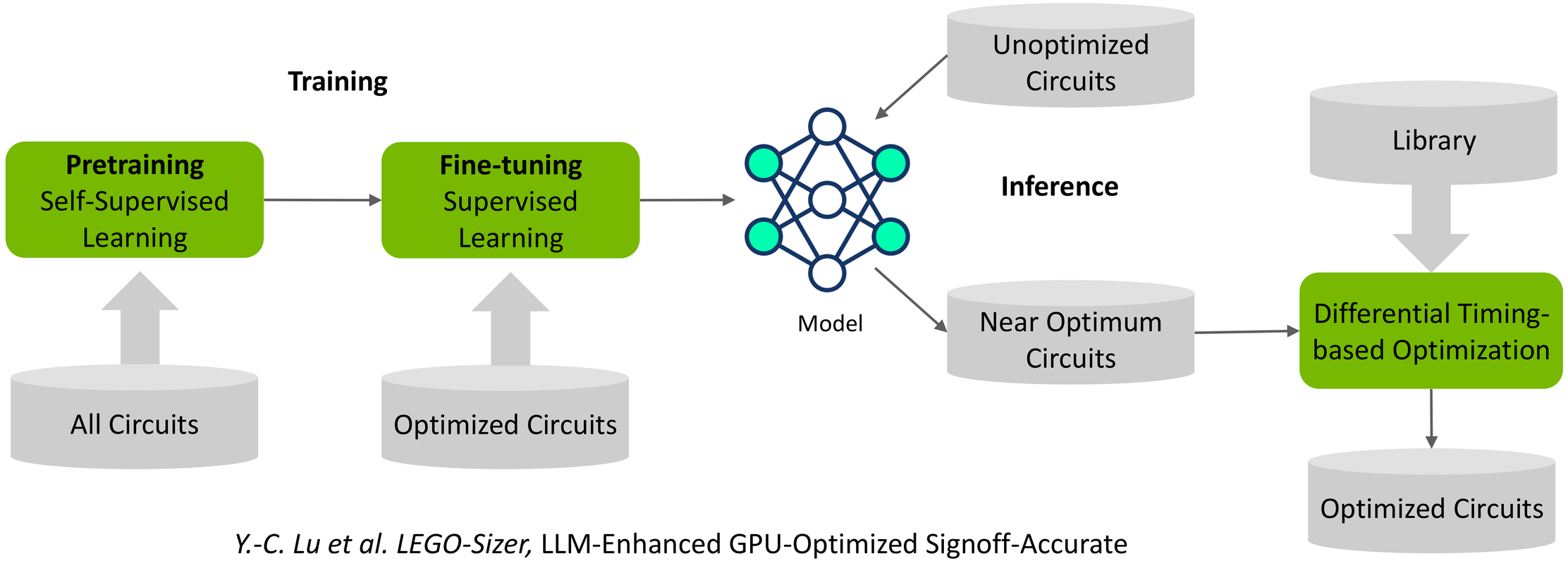


# LEGO-Sizer : 2nd attempt on GenAI for gate sizing

Encoder-only architecture predicts all gates on a path simultaneously

Similar to the pretraining/fine-tuning strategy, leverage unsupervised learning to learn the design representation and supervised learning to learn the optimization

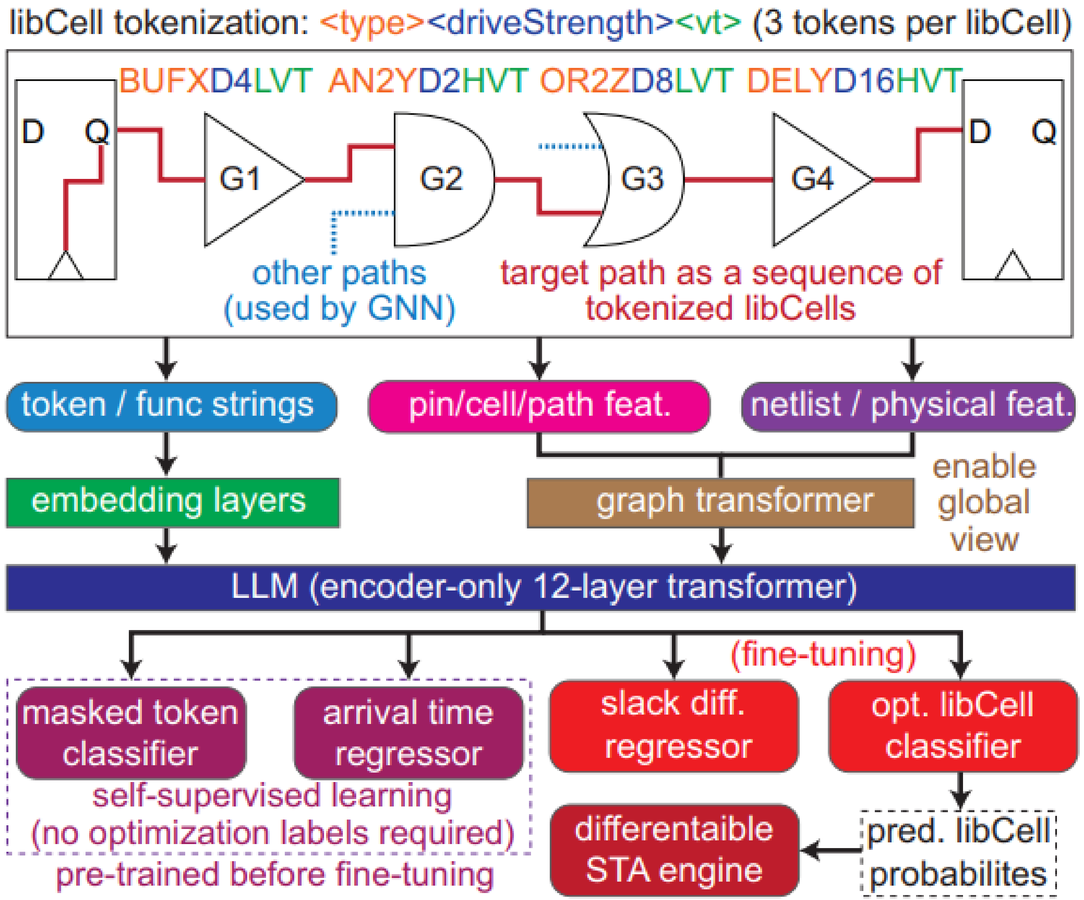
To fix the outliers after inference, run differential timing-based optimization




*Y.-C. Lu et al. LEGO-Sizer,* LLM-Enhanced GPU-Optimized Signoff-Accurate Differentiable VLSI Gate Sizing in Advanced Nodes, *ISPD'25*

# LEGO-Sizer Architecture

- Sophisticated embeddings to capture gate and netlist features
  - LLM-inspired tokenization for each gate
  - Timing table generated gate embeddings
  - Graph transformer to capture netlist features
- Pretraining tasks:
  - Masked token classifier
  - Arrival time regressor
- Fine-tuning tasks:
  - Slack difference regressor
  - Optimized libCell classifier
- Inference returns the initial probability of each libCell
- Differentiable STA engine built on PyTorch to optimize the probability of each libCell after inference



# LEGO-Sizer Results

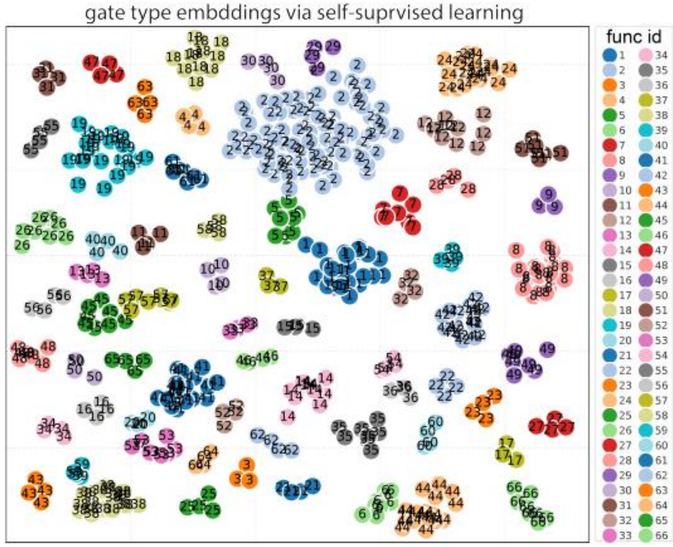
Pretrained on 22M timing paths over multiple 3nm blocks

Fine-tuned on ECO-optimized netlists

Inference on unseen designs from the fine-tuning

Better TNS/WNS optimization than ECO tools, >100X speedup

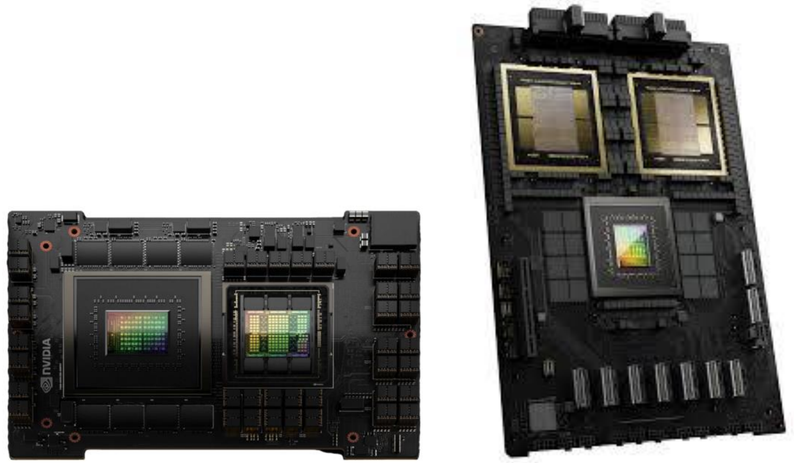
Learned embeddings show functional understanding




| unseen design (# cells) | initial state (pre-optimized) | | | | commercial signoff tool (32 threads) | | | | | LEGO-Size (GT + LLM + Differentiable STA) | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | WNS | TNS | #vio. EPs | total power | WNS | TNS (goal) | #vio. EPs | total power | speed-up | WNS | TNS (goal) | #vio. EPs | total power | speed-up |
| block1 (1.8M) | -0.066 | -17.46 | 2410 | 238.6 | -0.028 | -1.953 | 271 | 238.8 | 1.00 | -0.034 | -1.237 (-36.7%) | 184 (-47.3%) | 238.8 | 125x |
| block2 (1.3M) | -0.041 | -30.38 | 5410 | 293.4 | -0.038 | -7.82 | 1201 | 293.7 | 1.00 | -0.035 | -5.47 (-30.0%) | 943 (-27.4%) | 293.9 | 111x |
| block3 (1.2M) | -0.054 | -10.03 | 1539 | 119.7 | -0.032 | -5.37 | 906 | 119.8 | 1.00 | -0.033 | -4.32 (-19.5%) | 812 (-10.4%) | 119.8 | 100x |
| block4 (1.5M) | -0.102 | -59.25 | 6955 | 248.5 | -0.097 | -24.77 | 1912 | 249.1 | 1.00 | -0.083 | -20.36 (-17.8%) | 1658 (-15.3%) | 249.3 | 119x |
| block5 (1.4M) | -0.072 | -11.33 | 1525 | 127.2 | -0.038 | -6.68 | 620 | 127.5 | 1.00 | -0.036 | -4.85 (-27.4%) | 529 (-17.2%) | 127.5 | 114x |

# New EDA Computing Paradigm

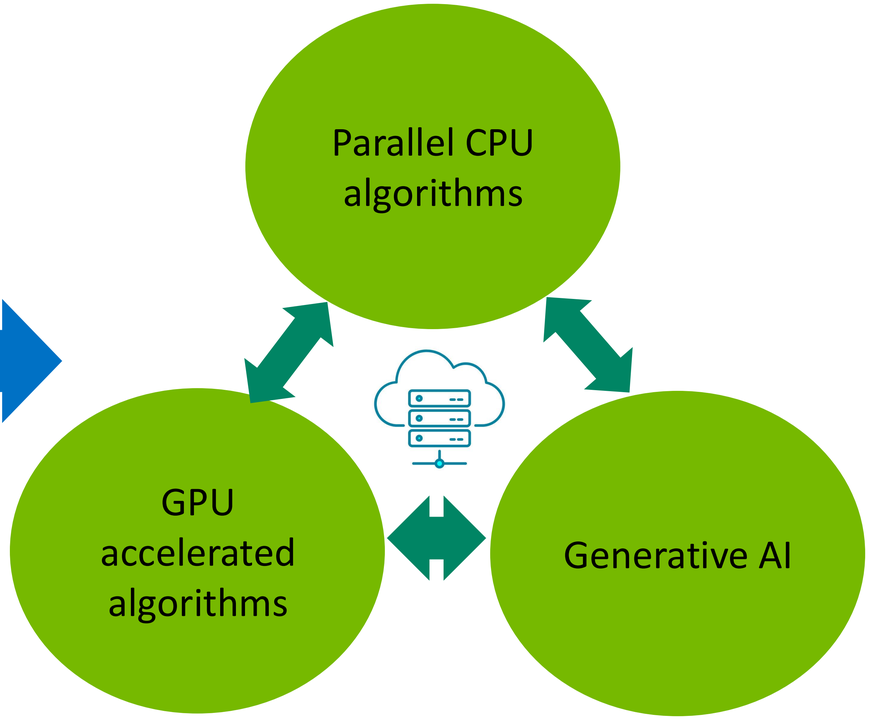
## Hardware fundamentally shapes software



GH and GB Superchips



Custom → Single thread CPU algorithms → Parallel CPU algorithms →



# Automate Chip Design with LLM

Used by thousands engineers in NVIDIA, saving hundreds engineer months/year

**Coding Assistance** → Generating design and verification code (RTL, SVA, testbenches, EDA scripts, tools scripts, and configs)

- Generate code from specifications
- Generate code for auxiliary design tasks such as assertions, comments, etc.
- Generate lower-level programs from higher-level descriptions
- Generate scripts for specific tasks (VLSI, Verification)
- Transform code for more efficient implementation

**Know-how Assistance** → Generating insights, knowledge, and ideas. Answer questions about designs, infrastructures, tools, flows, HW domains, etc.

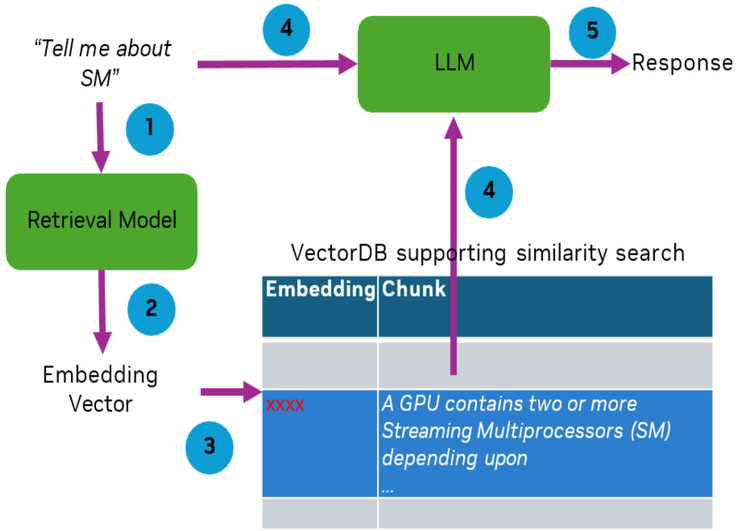
**Analysis Assistance** → Summarization, report/log analysis, visualization of design and related data, check rule violations, etc.

**Debug Assistance** → Triage and fix a design problem, e.g. regression failures for logic design and timing failures for physical design.

**Optimization Assistance** → Optimize for design and verification: coverage closure, recipe optimization, architecture exploration, etc.

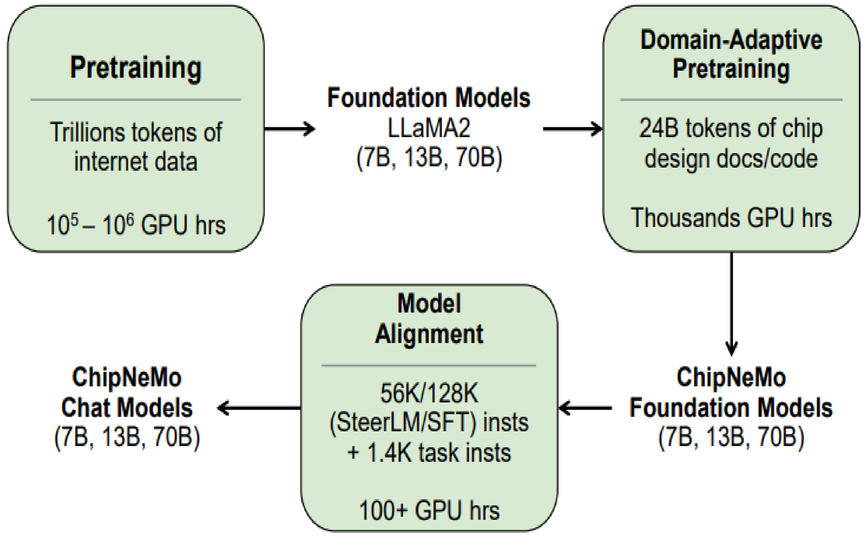
# Make LLM Learn to Do Chip Design

## In-Context Learning



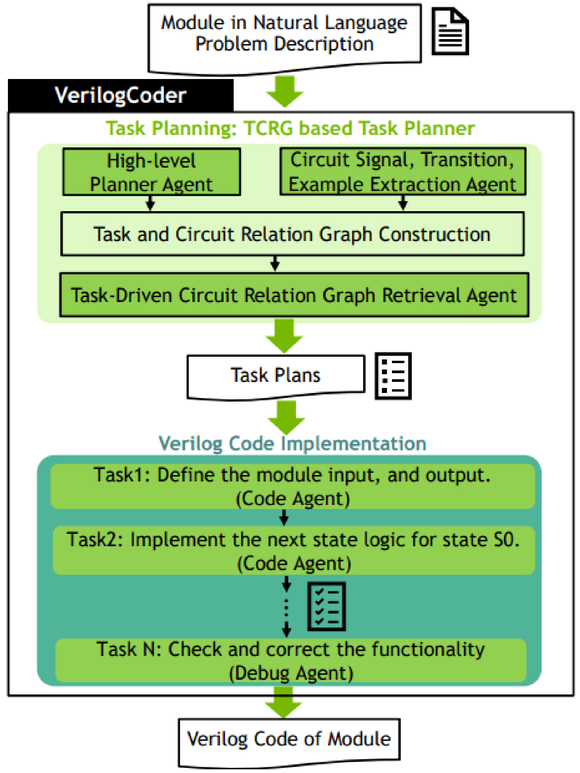

*Retrieval Augmented Generation (RAG)*

## Parameter Training




*M. Liu et al, ChipNeMo*

## Agent



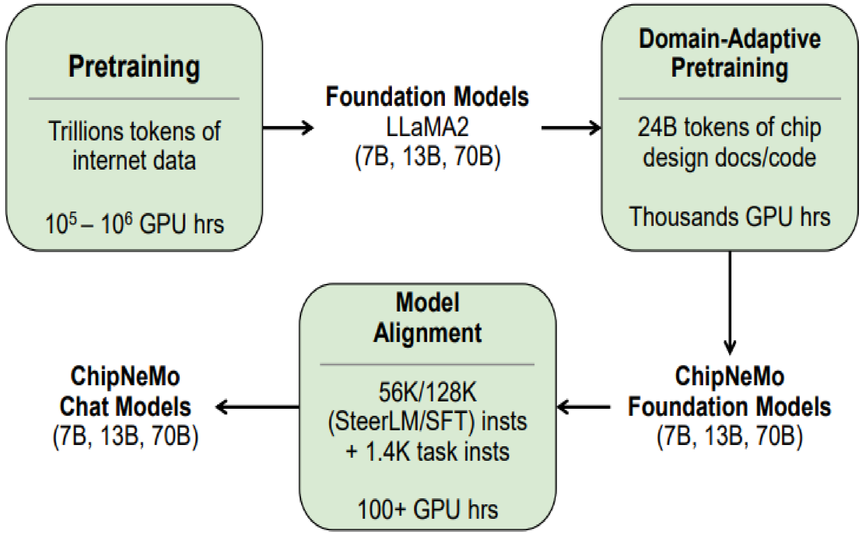

*C.-T. Ho et al, VerilogCoder*

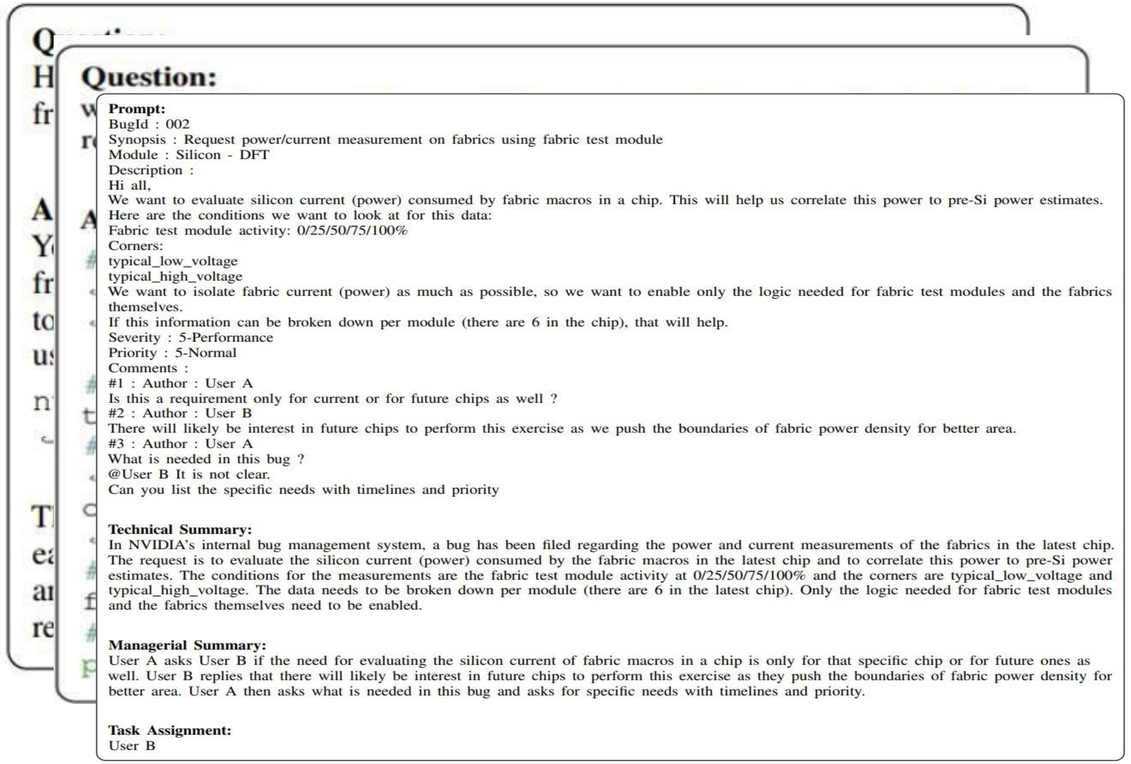
# ChipNeMo : domain-adapted model for chip design

Incorporate domain knowledge with continued pretraining using domain data

- Answer questions about designs, infrastructures, tools, flows, HW domains, etc.
- Generate scripts for specific tasks (VLSI)
- Summarize bug reports, predict assignment

Domain-adapted models have better performance



**Question:**

**Prompt:**
BugId : 002
Synopsis : Request power/current measurement on fabrics using fabric test module
Module : Silicon - DFT
Description :
Hi all,
We want to evaluate silicon current (power) consumed by fabric macros in a chip. This will help us correlate this power to pre-Si power estimates.
Here are the conditions we want to look at for this data:
Fabric test module activity: 0/25/50/75/100%
Corners:
typical_low_voltage
typical_high_voltage
We want to isolate fabric current (power) as much as possible, so we want to enable only the logic needed for fabric test modules and the fabrics themselves.
If this information can be broken down per module (there are 6 in the chip), that will help.
Severity : 5-Performance
Priority : 5-Normal
Comments :
#1 : Author : User A
Is this a requirement only for current or for future chips as well ?
#2 : Author : User B
There will likely be interest in future chips to perform this exercise as we push the boundaries of fabric power density for better area.
#3 : Author : User A
What is needed in this bug ?
@User B It is not clear.
Can you list the specific needs with timelines and priority

**Technical Summary:**
In NVIDIA's internal bug management system, a bug has been filed regarding the power and current measurements of the fabrics in the latest chip. The request is to evaluate the silicon current (power) consumed by the fabric macros in the latest chip and to correlate this power to pre-Si power estimates. The conditions for the measurements are the fabric test module activity at 0/25/50/75/100% and the corners are typical_low_voltage and typical_high_voltage. The data needs to be broken down per module (there are 6 in the latest chip). Only the logic needed for fabric test modules and the fabrics themselves need to be enabled.

**Managerial Summary:**
User A asks User B if the need for evaluating the silicon current of fabric macros in a chip is only for that specific chip or for future ones as well. User B replies that there will likely be interest in future chips to perform this exercise as they push the boundaries of fabric power density for better area. User A then asks what is needed in this bug and asks for specific needs with timelines and priority.

**Task Assignment:**
User B

*M. Liu et al, ChipNemo: Domain-Adapted LLMs for Chip Design*

# Limitations of Domain-Adapted LLMs

## Lack of critical instruction following capabilities after domain-adapted training

While having strong domain knowledge, ChipNeMo fails to follow various users' instructions

### Prompt

List **exactly 10** possible names for a new baby boy. Then add a postscript **starting with P.P.S** to the end of your response. Put the names into bullet points that are indicated by:**\n*** This is an example bullet.

### ChipNeMo

\n1. John
\n2. James
\n3. Robert
\n4. Michael
\n5. William
\n6. David
\n7. Richard
\n8. Charles
\n9. Joseph
\n10. Thomas

P.P.S. I hope you find a name you like!

### Llama2-70B-Chat

Sure, here are ten possible names for a new baby boy:

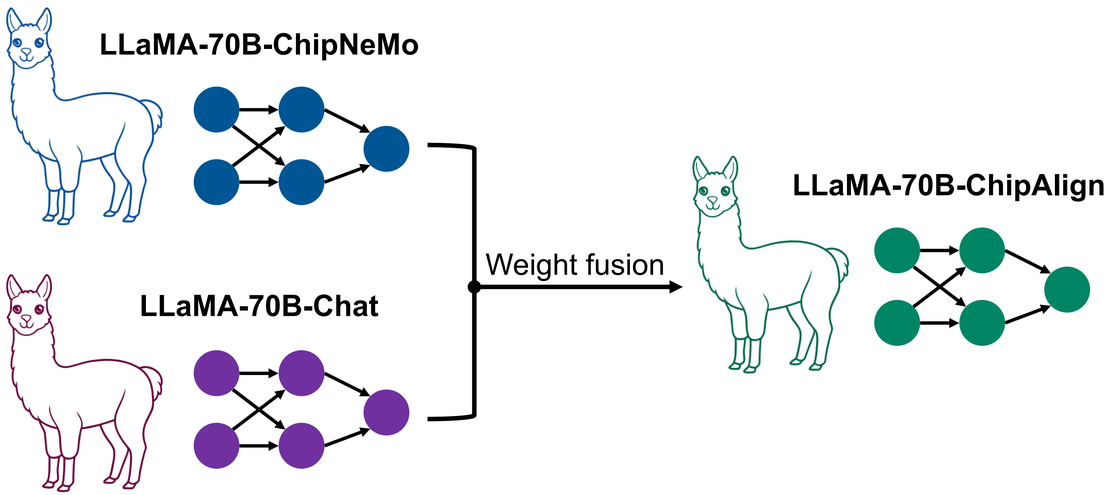
\n* Liam
\n* Noah
\n* Ethan
\n* Logan
\n* Lucas
\n* Benjamin
\n* Mason
\n* Oliver
\n* Alexander
\n* Elijah

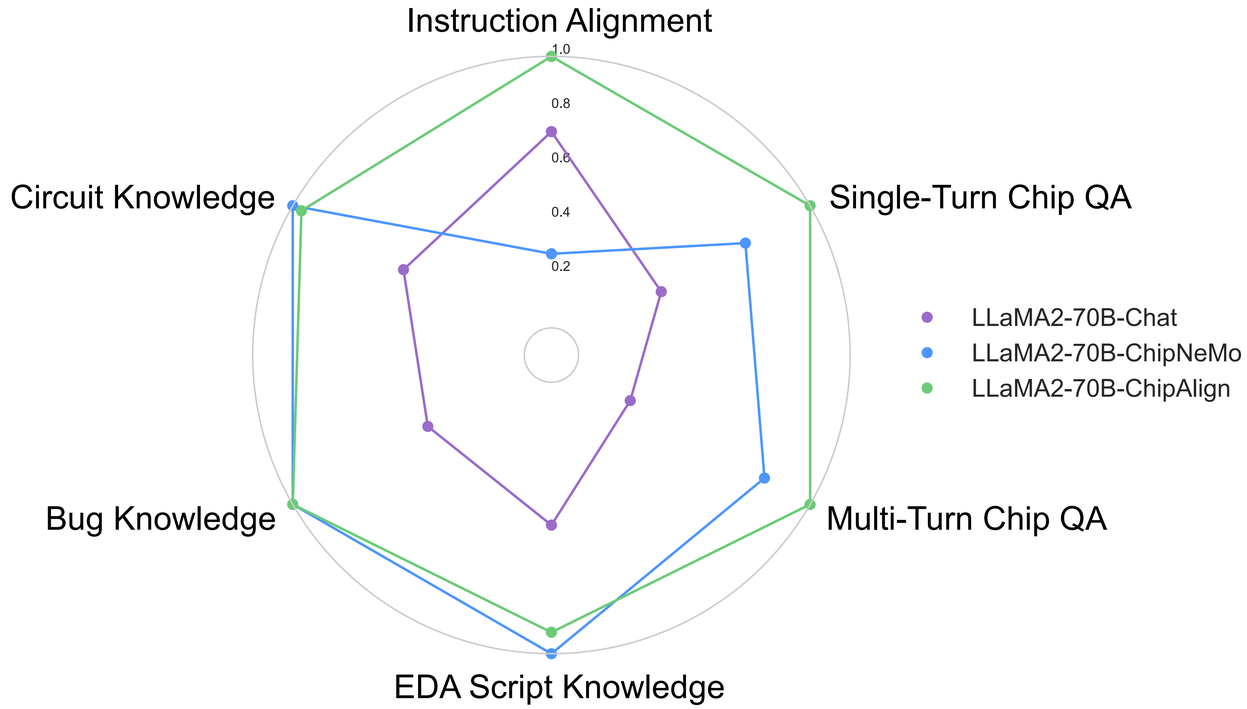
P.P.S. I hope these names meet your expectations! Let me know if you need any further assistance.

# ChipAlign: Model Merging for Better Instruction Alignment

High-level idea:

- Merging weights of two LLMs that are good at different tasks (e.g., instruction following and hardware knowledge)
- The merged LLM is then good at both tasks




*C. Deng et al, ChipAlign: Instruction Alignment in Large Language Models for Chip Design via Geodesic Interpolation, DAC '25*

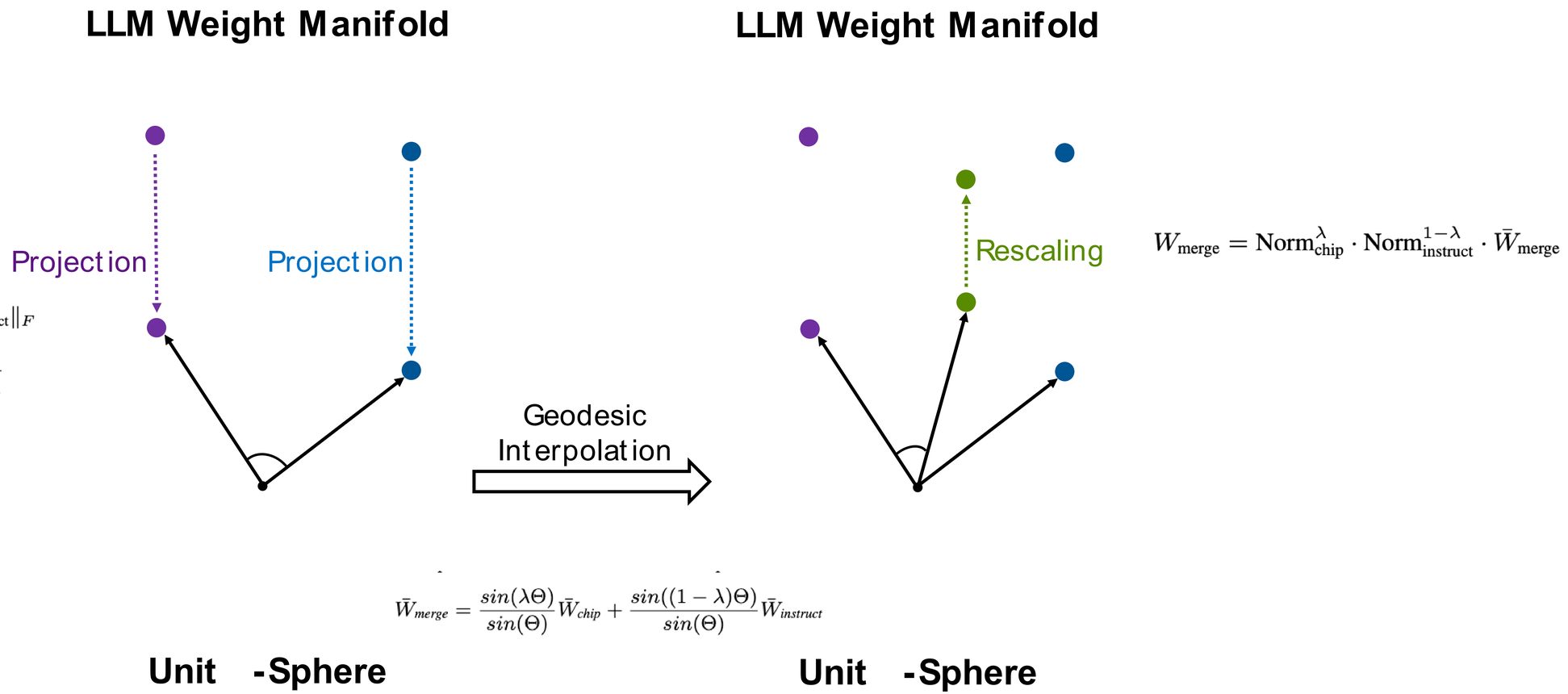
# Model Merging with Geodesic Interpolation

Interpolate between two LLMs to produce the merged LLM

- Interpolate along the shortest path on the “weight manifold”
- $\lambda$ is a hyperparameter

$$\text{Norm}_{\text{chip}} = \|W_{\text{chip}}\|_F, \quad \text{Norm}_{\text{instruct}} = \|W_{\text{instruct}}\|_F$$

$$\bar{W}_{\text{chip}} = \frac{W_{\text{chip}}}{\text{Norm}_{\text{chip}}}, \quad \bar{W}_{\text{instruct}} = \frac{W_{\text{instruct}}}{\text{Norm}_{\text{instruct}}}$$




$$W_{\text{merge}} = \text{Norm}_{\text{chip}}^{\lambda} \cdot \text{Norm}_{\text{instruct}}^{1-\lambda} \cdot \bar{W}_{\text{merge}}$$

$$\bar{W}_{merge} = \frac{sin(\lambda\Theta)}{sin(\Theta)}\bar{W}_{chip} + \frac{sin((1-\lambda)\Theta)}{sin(\Theta)}\bar{W}_{instruct}$$

# LLaMA-70B-ChipNeMo Results

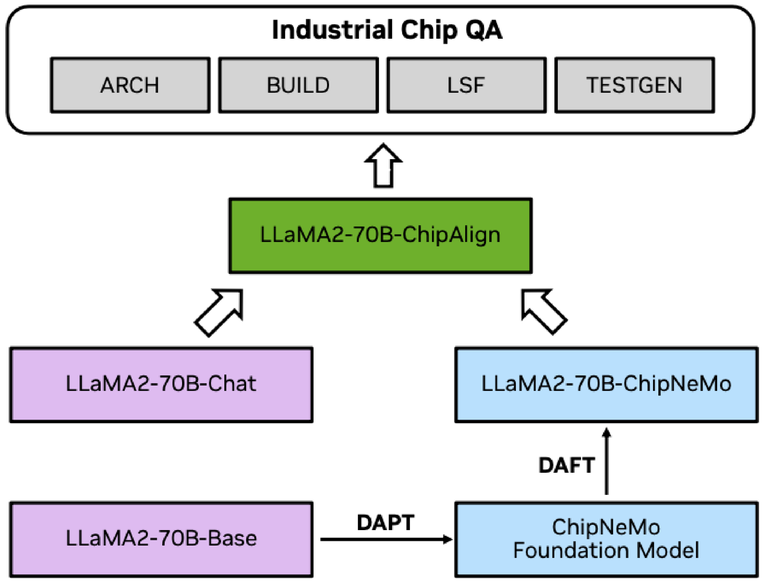



Table 2: GPT4-aided evaluation scores on industrial production-level chip QA benchmarks.

| | Single Turn | | | | | Multi Turn | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| **Method** | ARCH | BUILD | LSF | TESTGEN | All | ARCH | BUILD | LSF | TESTGEN | All |
| LLaMA2-70B-Chat | 40.00 | 70.50 | 54.25 | 66.75 | 57.00 | 10.00 | 41.00 | 10.50 | 50.00 | 30.25 |
| LLaMA2-70B-ChipNeMo | 45.00 | 81.75 | 60.50 | 87.50 | 66.75 | 40.00 | **81.75** | 52.00 | 33.25 | 54.50 |
| LLaMA2-70B-ChipAlign | **57.50** | **84.00** | **70.75** | **91.75** | **74.25** | **42.50** | 75.00 | **60.50** | **79.25** | **62.75** |

# Agentic systems make LLMs much more powerful

**Non-Agentic**

Simple Question

LLM

Answer

**Agentic**

Complex Question

Planning

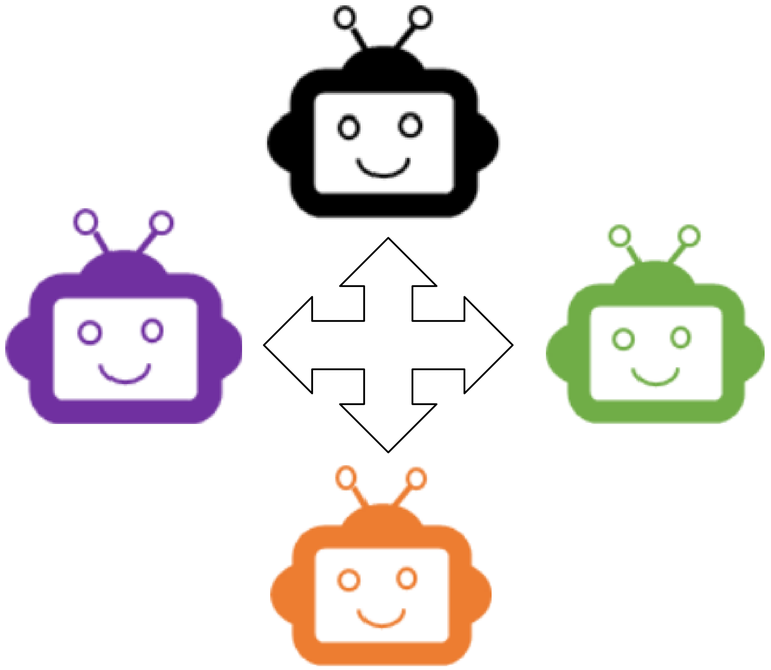
Memory

Tools

LLM

Answer

Single Agent



Multi-Agent

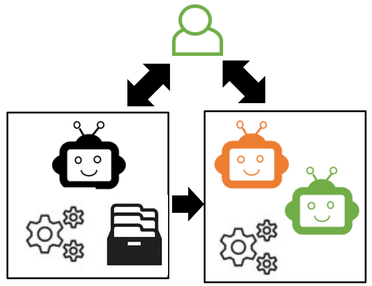
# Progression of Our Agent Research for Hardware Design

Agents

More Agent for Hardware Design

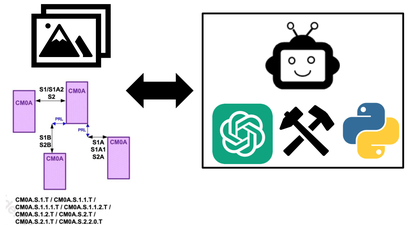
**SPICE Verification Agent**
**ChipVQA Agent**
**ERC Check Agent**
**Formal Verification Agent**
**SPEC2RTL Agent**
⋮

**MCMM Timing Analysis**
Multi-AIs + Dynamic Task Graph + Human Interactive

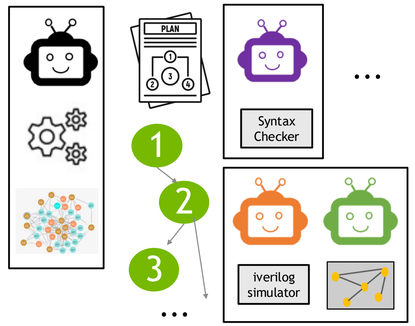


≈ 60× Speedups

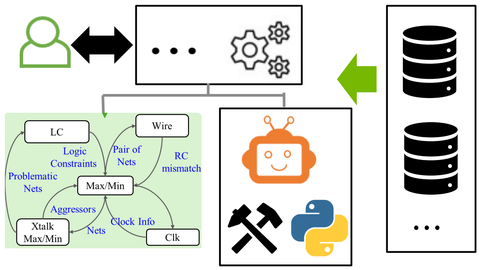
**DRC-Coder [ISPD 2025]**
Multi-Modality & Multi-AIs + Tools + Coding



**VerilogCoder [AAAI 2025]**
Multi-Ais + Dynamic Task Graph + Customized Debug Tools

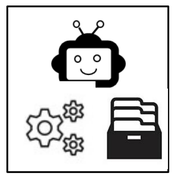



**Timing Path Debug**
Static Task Graph + Multi-AIs + Agentic Retrieval

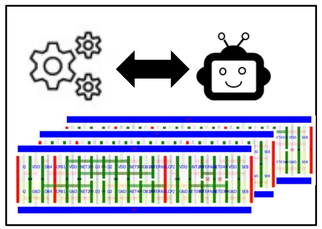



Reduce hours to mins

**RTLFixer [DAC'24]**
ReAct+ Compiler + RAG



**LLM for Standard Cell Design Optimization [LAD'24 Best Paper]**
ReAct+ Netlist Tools Reasoning



Sept, 2023

April, 2024

July, 2024

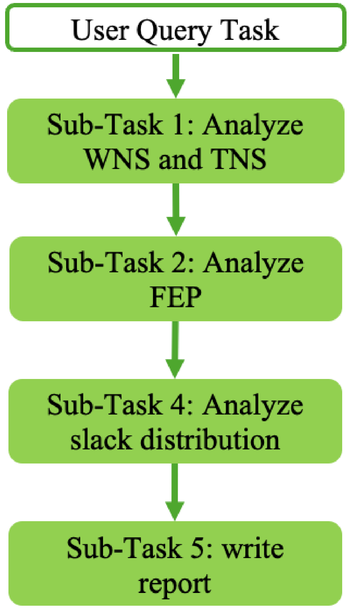
# Agent for Hardware Design Tasks

How to acquire domain capability and improve deployment efficiency

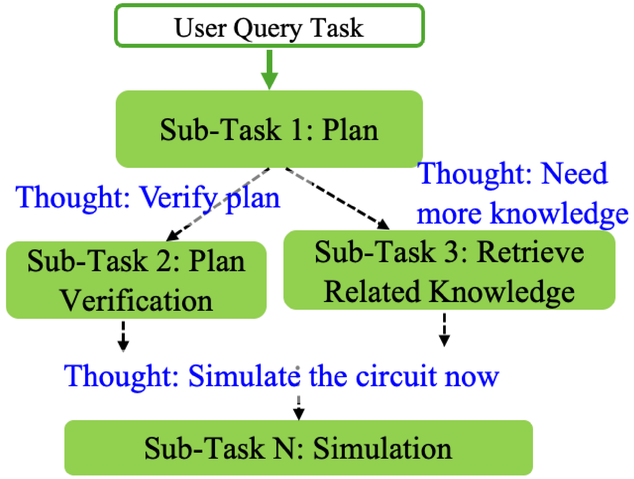
- Extensive domain capability required to build useful agents for chip design
  - Diverse design knowledge: Logic design, physical design, verification, analog design, etc.
  - Diverse set of tool commands and file format: PrimeTime, ICC2, Innovus, reports, LEF/DEF, etc
- Capability acquisition methods
  - By model: foundation and fine-tuned models
  - By prompt: prompt engineering
  - By tools: RAG/customized tools/APIs
  - By decomposition: task flow, multi-Agents conversation
  - By learning: trial-and-error, experience accumulation
- Domain knowledge comes from the designer's insights
- Need efficient framework for expert designers to deploy agent using off-the-shelf models

# Marco Agent Framework

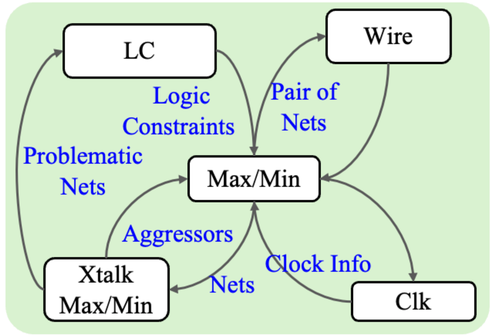
- Move from *reactive* ReAct agent (ad-hoc) to hierarchical graph-based task planning (structured) : task flow + sub-task : like driving (route planning first, then drive by visual) or System 2 vs 1
- Incorporating design knowledge in task flow planning
  - **Structure**: Static vs Dynamic vs Search
  - **Knowledge**: Predefined vs Reasoning vs Retrieval-based
- Solve sub-task with *reactive* reasoning: Single Agent (ReAct) or Multi-Agent conversation



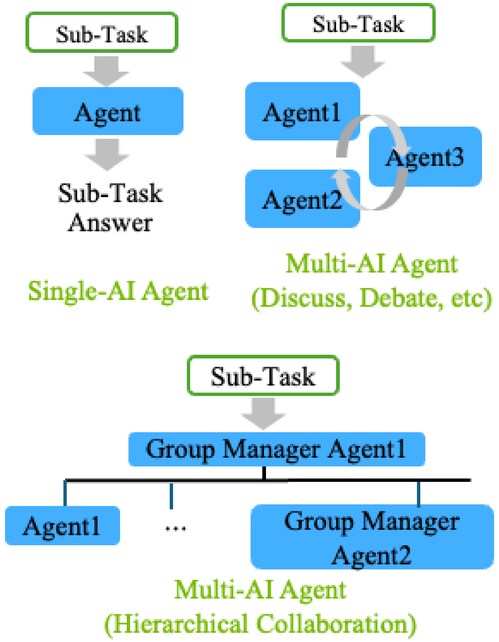

**Static task flow**




**Dynamic task flow graph**




**Knowledge graph for task planning**

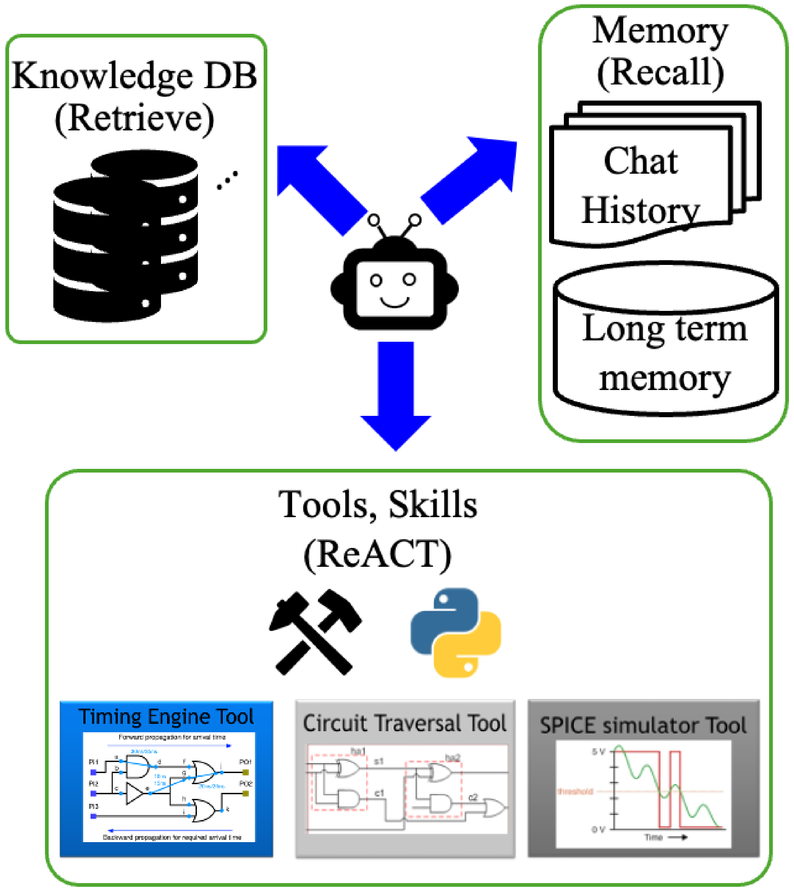



**Solving Sub-tasks**

*C.-T. Ho et al., "Marco: Configurable Graph-Based Task Solving and Multi-AI Agents Framework for Hardware Design", VLSI'25*

# Customized Tools for HW Agents

| Agents | Task Category | Customized Tools |
|---|---|---|
| RTLFixer | Code Syntax Fixing | RTL Syntax Error RAG Database |
| Standard Cell Layout Opt. | Optimization | Cluster Evaluator, Netlist Traverse Tool |
| MCMM Timing Analysis (Partition/Block-Level) | Summary & Anomaly Identification | Timing Distribution Calculator, Timing Metric Comparator |
| Timing Path Debug (Path-Level) | Summary & Anomaly Identification | Agentic Timing Report Retrieval |
| DRC Coder | Code Generation | Foundry Rule Analysis, Layout DRV Analysis, DRC Code Evaluation |
| VerilogCoder | Code Generation | TCRG Retrieval Tool, AST-Based Waveform Tracing Tool |
| FVAgent | Code Generation | Rule RAG Database, FV tools |
| Spec2RTL Agent | Code Generation | HLS tools |



# VerilogCoder: Solving VerilogEval benchmark

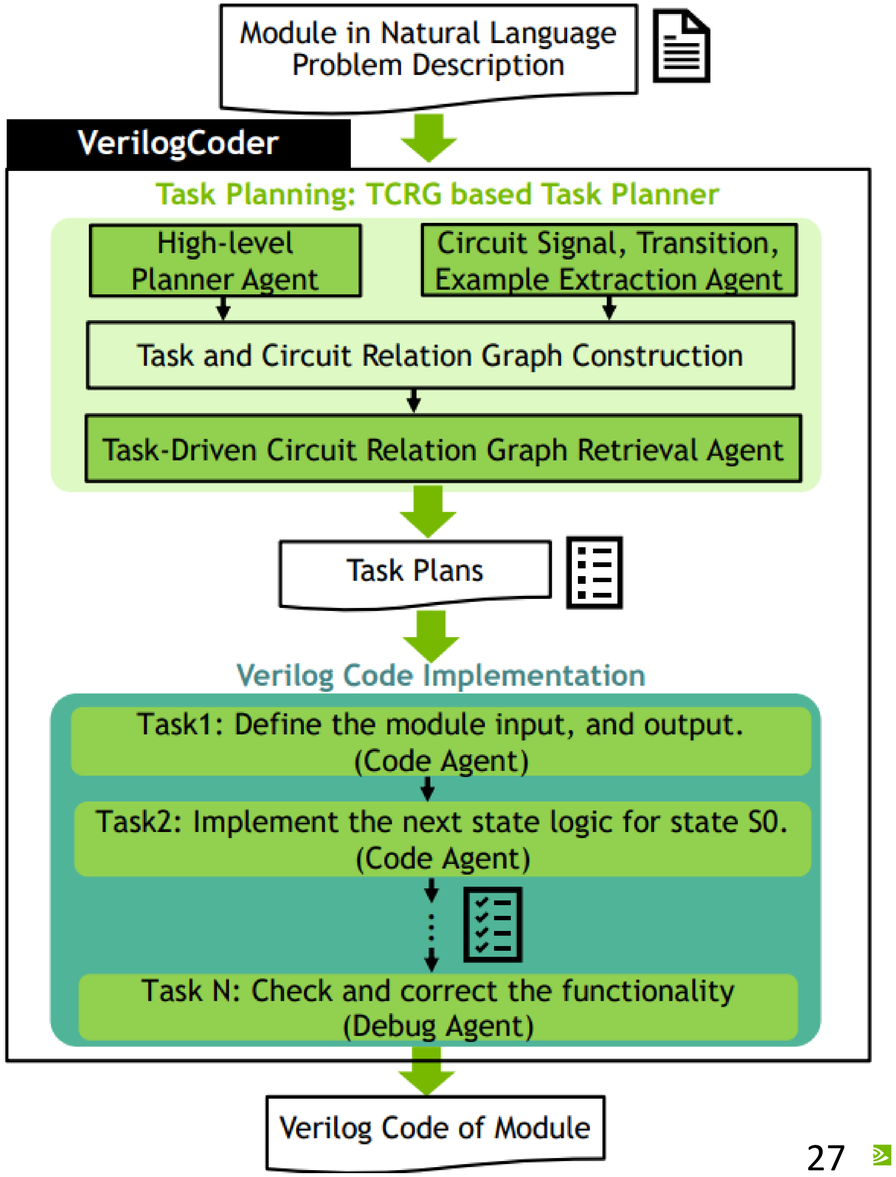
https://github.com/NVlabs/VerilogCoder

Two major stages:

The **task planning** stage generates a task plan

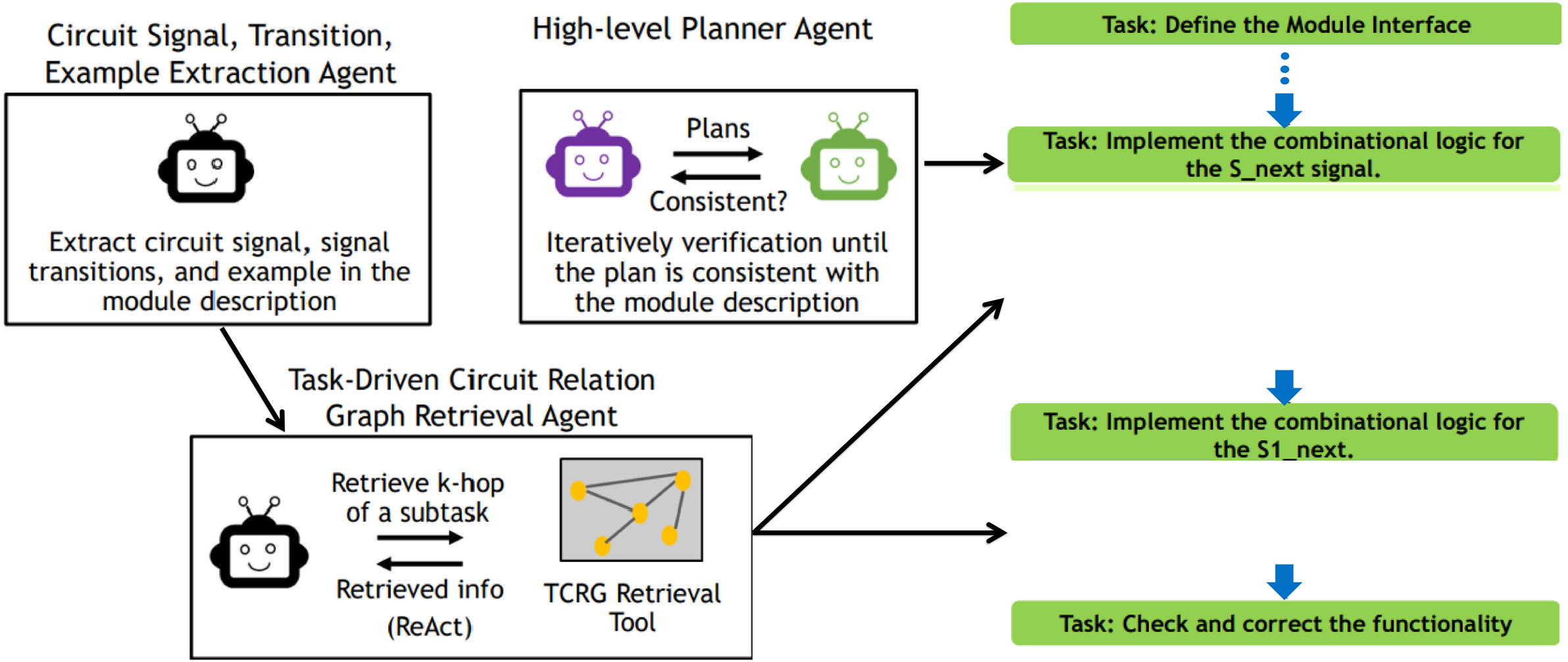
The **code implementation** stage writes code for each planned task and debug the code

Dynamic task plan generation and execution




*C.-T. Ho et al, VerilogCoder: Autonomous Verilog Coding Agents with Graph-based Planning and Abstract Syntax Tree (AST)-based Waveform Tracing Tool*

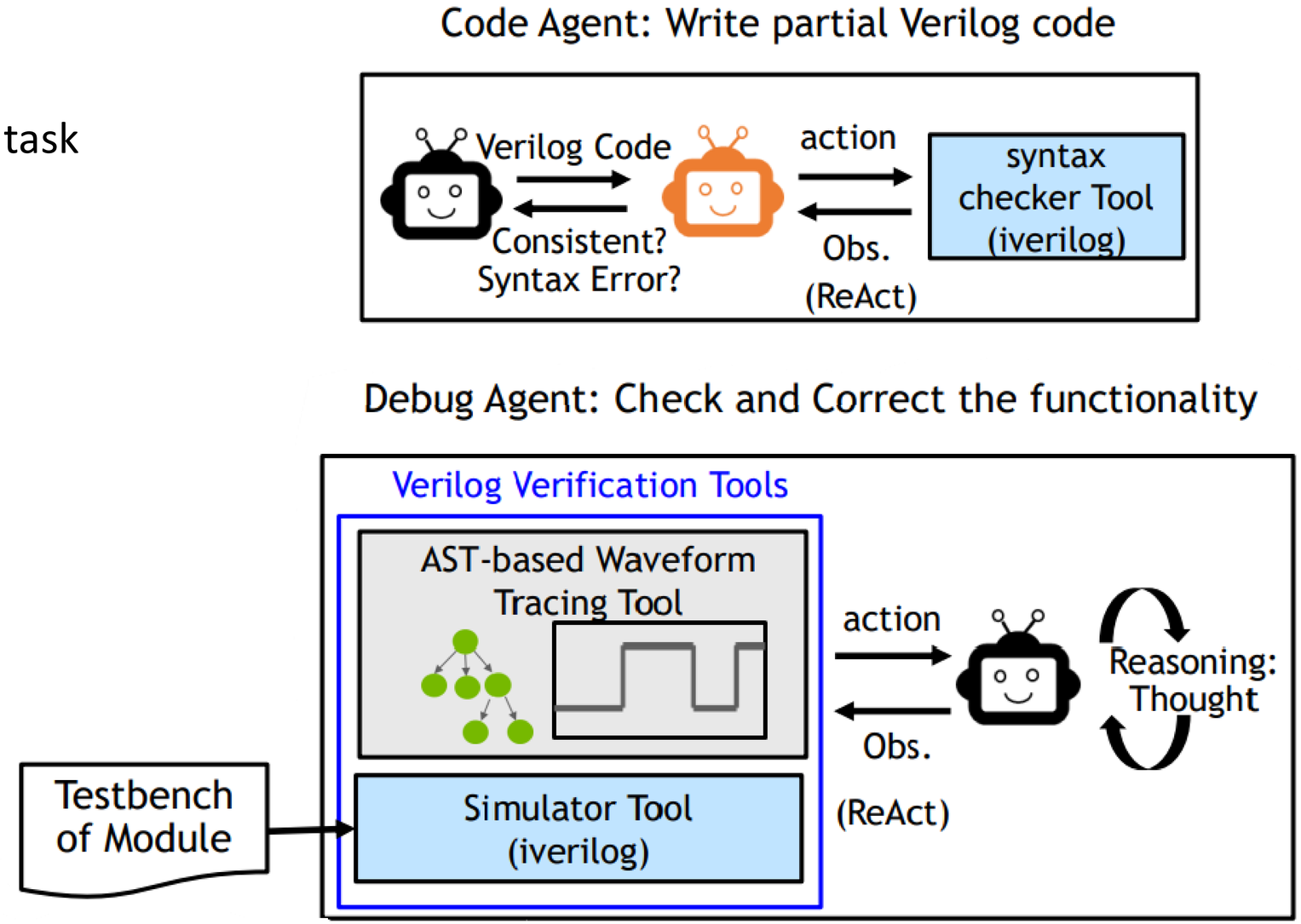
# Multi-agent planner builds Knowledge Graph to extract detailed information for task planning



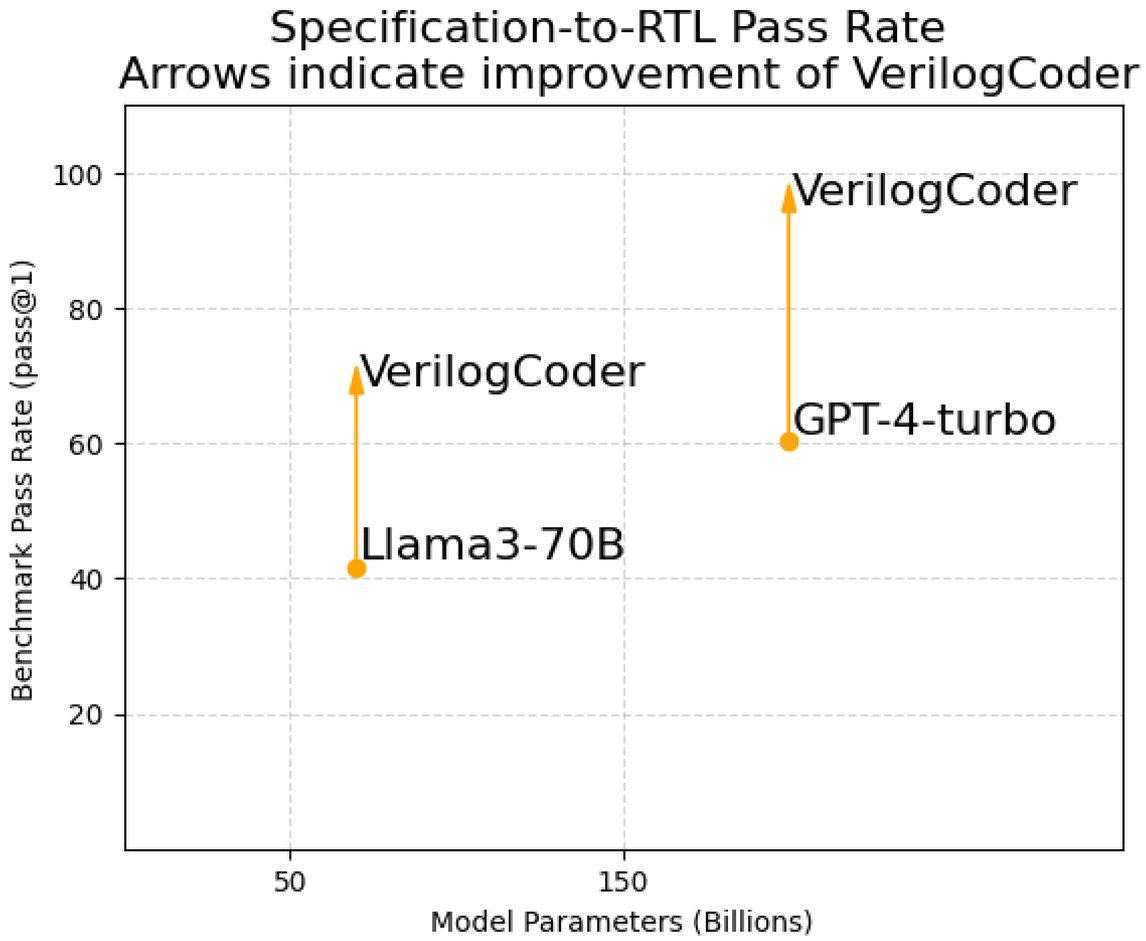
# Multi-agent coder leverages syntax checker and waveform tracing tools to write and debug code

Writing code for each task

Debug final code



# VerilogCoder solves most of problems in VerilogEval



Mate Terminal | Mate Terminal | Mate Terminal

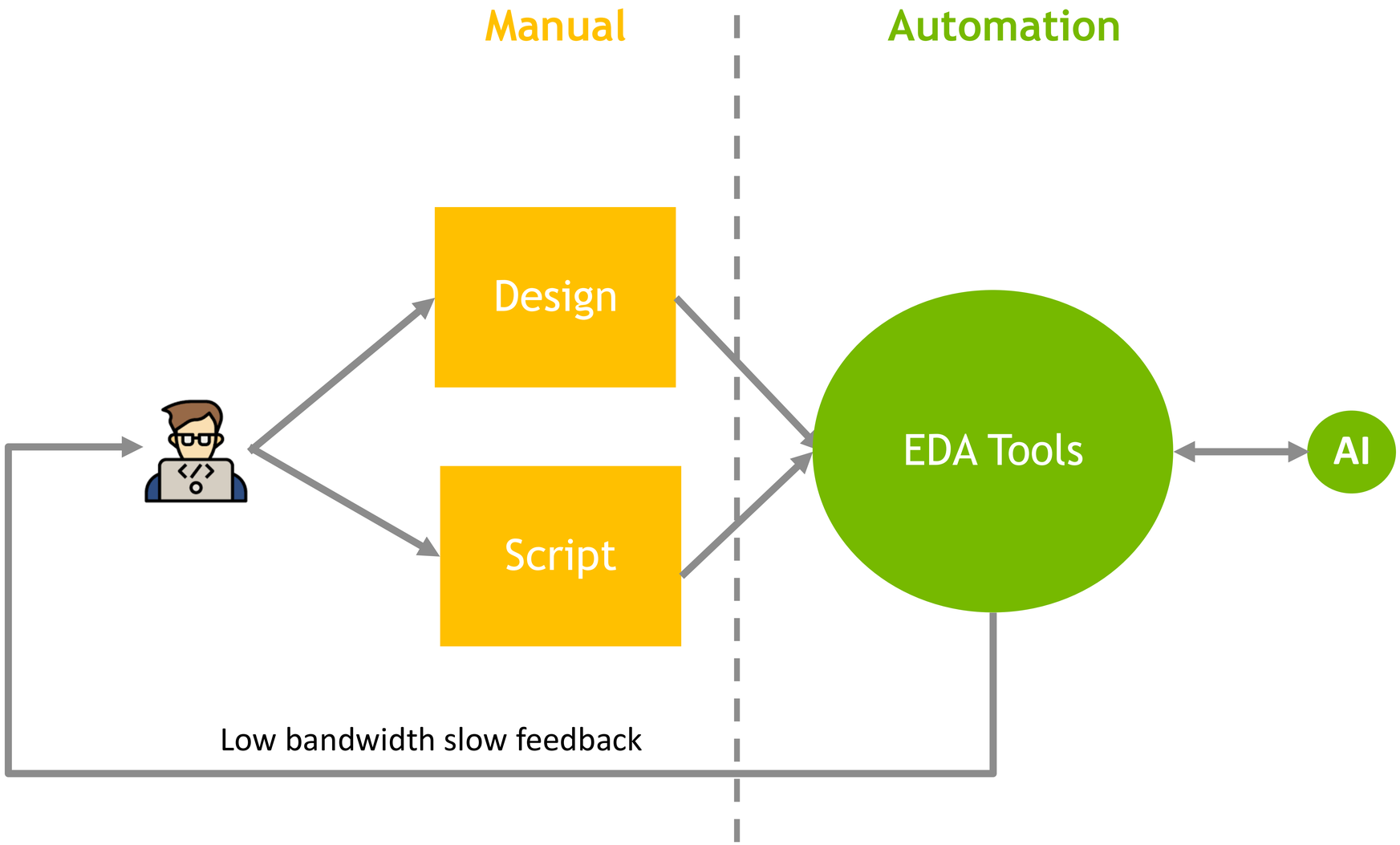
```
(/home/scratch.chiatungh_nvresearch/nvcell_env/llm_env) bash-4.2$ python hardware_agent/examples/verilog_eval_agent/Verilog_eval_task_flow_ver1.py
reading  /home/scratch.chiatungh_nvresearch/hardware-agent-marco/hardware_agent/examples/verilog_testcases/verilog-eval-v2/dataset_dumpall/ Prob139_2013_q2bfsm_prompt.txt
reading  /home/scratch.chiatungh_nvresearch/hardware-agent-marco/hardware_agent/examples/verilog_testcases/verilog-eval-v2/dataset_dumpall/ Prob139_2013_q2bfsm_ref.sv
reading  /home/scratch.chiatungh_nvresearch/hardware-agent-marco/hardware_agent/examples/verilog_testcases/verilog-eval-v2/dataset_dumpall/ Prob139_2013_q2bfsm_test.sv
{'llm_class': <class 'adlrchat.langchain.LLMGatewayChat'>, 'llm_kwargs': {'model_name': 'gpt-4', 'temperature': 0.0, 'top_p': 1.0}}
case manager length =  1
current test is  2013_q2bfsm
current task id is  2013_q2bfsm
current test is  2013_q2bfsm
register tool  sequential_flipflop_latch_identify_tool <function sequential_flipflop_latch_identify_tool at 0x7f65abdac040>  caller:  <autogen.agentchat.assistant_agent.AssistantAgent object at 0x7f65a93ad1b0>  executer:  <autogen.agentc
hat.user_proxy_agent.UserProxyAgent object at 0x7f65a93ad390>
Hardware Agent Initialized 1 proxy, 0 rag proxy, 2 assistants
Hardware Agent Initialized 1 proxy, 0 rag proxy, 1 assistants
user (to chat_manager):

You are a Verilog RTL designer that can break down complicated implementation into subtasks implementation plans.

[Example Begin]

### Problem

I would like you to implement a module named TopModule with the following
interface. All input and output ports are one bit unless otherwise
specified.

 - input  clk
 - input  reset
 - input  in (8 bits)
 - output out (8 bits)

The module should implement an 8-bit registered incrementer. The 8-bit
input is first registered and then incremented by one on the next cycle.

Assume all sequential logic is triggered on the positive edge of the
clock. The reset input is active high synchronous and should reset the
output to zero.

### Solution

module TopModule
(
  input  logic       clk,
  input  logic       reset,
  input  logic [7:0] in,
  output logic [7:0] out
);

  // Sequential logic

  logic [7:0] reg_out;

  always @( posedge clk ) begin
    if ( reset )
      reg_out <= 0;
    else
      reg_out <= in;
  end

  // Combinational logic

  logic [7:0] temp_wire;

  always @(*) begin
    temp_wire = reg_out + 1;
```

# Agent Research Challenges and Opportunities

- Engineering problems are diverse. It is not scalable to maintain customized agents for each use case
  - How to make a general-purpose agent?
- EDA tools are proprietary and often controlled by Tcl language
  - How to make agent learn to use these tools efficiently
- Engineering knowledge is often in designer's head not on any text
  - How to learn from designer feedback to improve agent
- Agent only follows engineer's guidance
  - Can Agent self-evolve to become super human
- Solving grand challenges such as design verification
- Improving design PPA

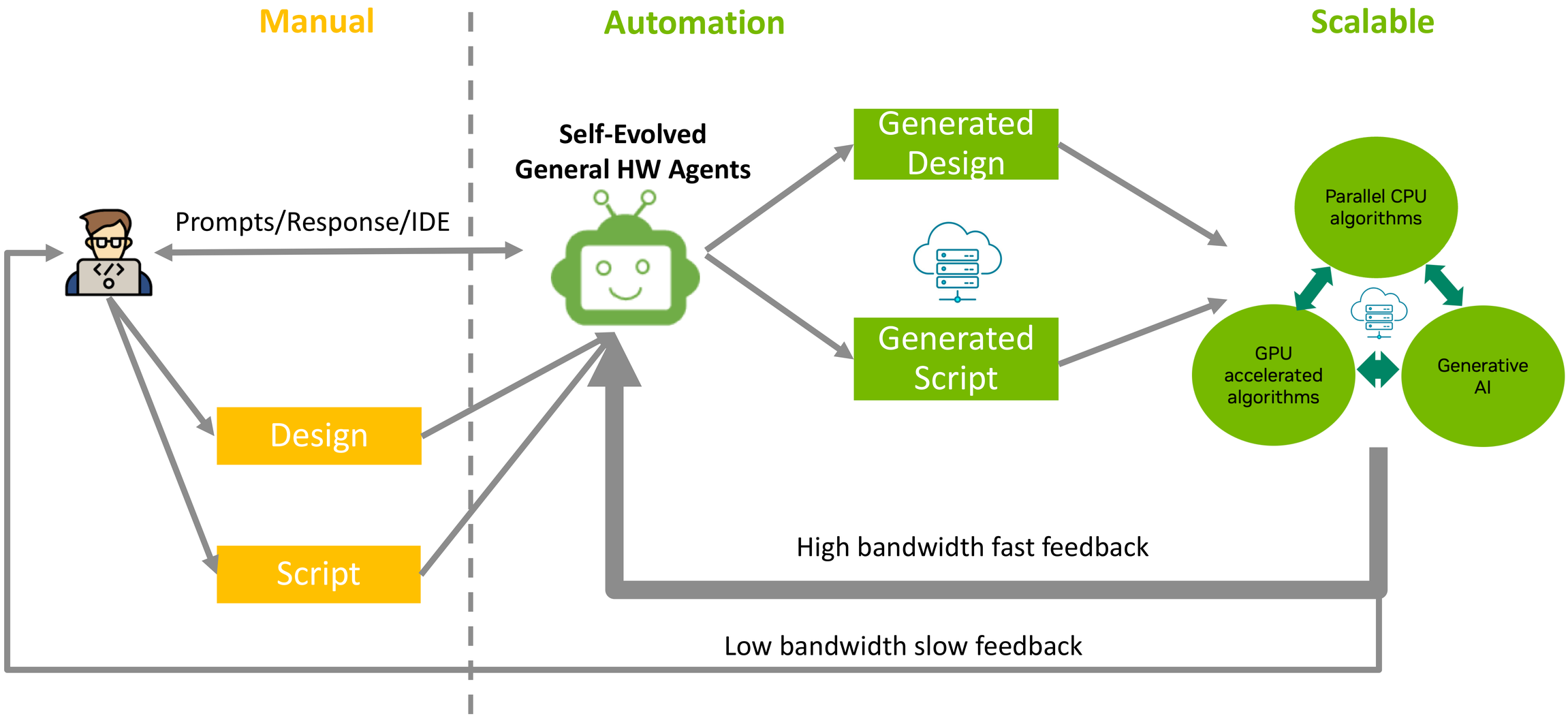
# Design Methodology Today

Lots of Manual efforts, unscalable EDA tools



# Design Methodology Tomorrow with AI

## More Automation, More Scalable



# Acknowledgments


## NVIDIA Design Automation Research Group

Haoxing (Mark) Ren

Team Leader

Chenhui Deng

Graph Learning for EDA, LLM for Chip Design

Chia-Tung (Mark) Ho

LLM Agent for Chip Design, Machine Learning for VLSI, DTCO, Standard Cell Layout Automation

Danny Liu

Reinforcement Learning, Machine Learning for EDA, Generative AI

Haoyu Yang

AI/LLM for EDA, GPU-accelerated EDA, AI for MultiPhysics Simulation, AI for Semiconductor Manufacturing

Rongjian Liang

Machine Learning for EDA, EDA for Machine Learning

Wen-Hao Liu

Circuits and VLSI Design

Yanqing Zhang

GPU Accelerated EDA, Machine Learning for EDA, GPU Accelerated Simulation

Yi-Chen Lu

Physical Design, Machine Learning for EDA

Yunsheng Bai

Formal Verification, Machine Learning for EDA

## Also Contributions from Interns and Collaborators


*To learn more about our research, check out https://research.nvidia.com/labs/electronic-design-automation/*